



分布与并行数据库

10 并行数据库简介

主讲教师：鄂金龙
中国人民大学信息学院



主要内容

- 并行数据库系统概念
- 硬件体系结构
- 数据分片技术
- 并行性种类
- 并行连接
- 并行排序
- 并行聚合

并行数据库系统概念

- 为什么并行存取数据？

Why Parallel Access To Data?

At 10 MB/s
1.2 days to scan

1,000 x parallel
1.5 minute to scan.



1 Terabyte

10 MB/s

Parallelism:
divide a big problem
into many smaller ones
to be solved in parallel.



并行数据库系统概念

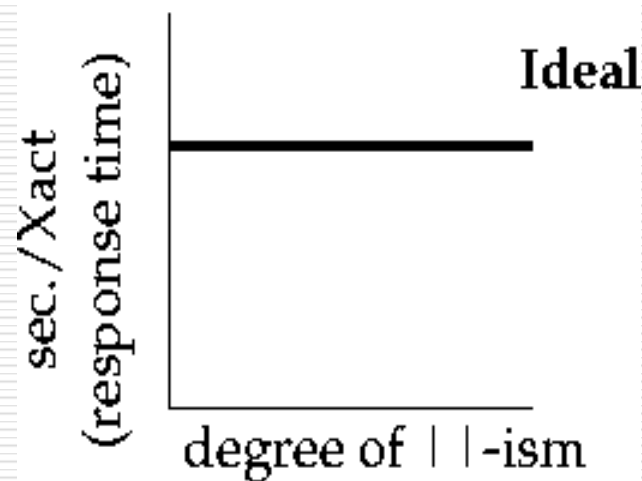
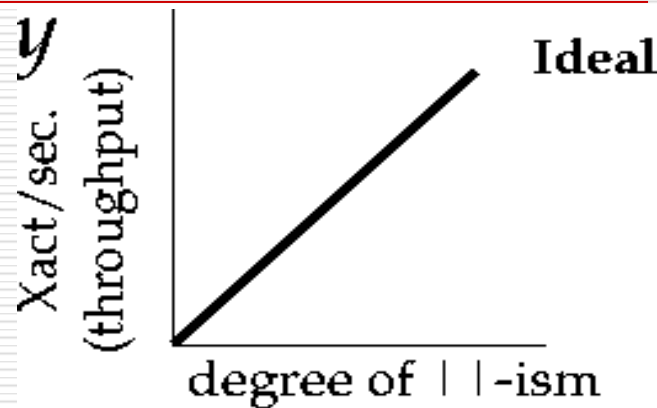
- 为什么并行存取数据？
 - 数据密集型(data-intensive)应用，如决策支持系统、在线处理分析(OLAP)、数据仓库(data warehouse)、知识和数据发现(KDD)等
- 并行数据库系统设计的研究问题：
并行I/O、并行查询优化、并行性数据库操作等

并行数据库系统概念

- 并行数据库系统的评价参数:

(1) speedup: 对于某个固定的计算任务, 1倍计算资源系统所完成的时间与n倍计算资源所完成时间之比; 理想的speedup曲线为线性加速

(2) scaleup: 1倍计算任务在1倍计算资源系统所完成的时间与n倍计算任务在n倍计算资源系统所完成时间之比, 理想的scaleup曲线为 $y=1$



并行数据库系统概念

- 影响并行数据库系统性能的三个因素
 - (1)启动代价(startup): 启动一个并行操作的代价
 - (2)干扰(interference): 共享资源之间的相互干扰
 - (3)倾斜(skew): 一个操作或一个查询可以被分解为若干个可并行执行的子操作或子查询, 执行时间最长的子操作或子查询的响应时间决定该并行操作的响应时间

并行数据库系统概念

- 实现并行的2种基本技术

- (1)管道

- 一个操作的输出是另一个操作的输入

- (2)分片

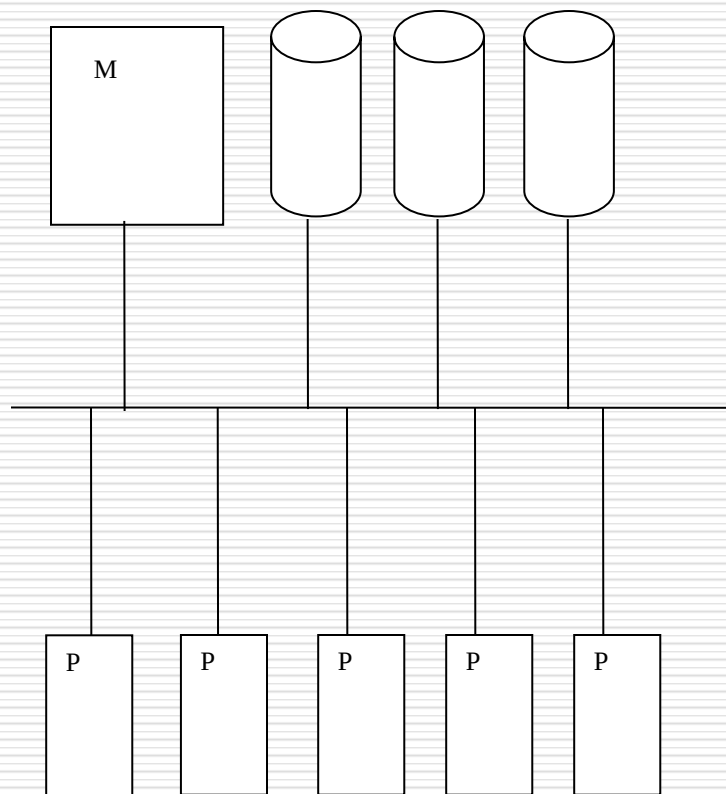
- 多台机器在不同的数据分片上做相同的事情



硬件体系结构

(1) 共享内存(SM, SE)

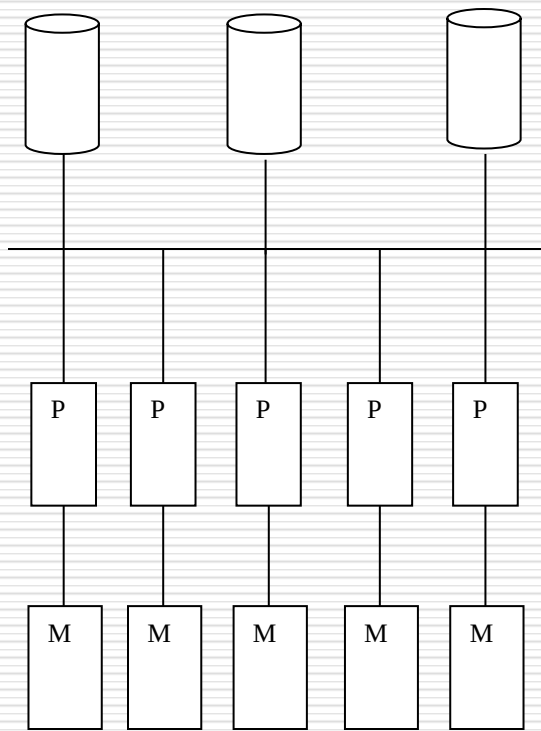
- ① 在**SM**体系结构中，处理机和磁盘可以通过一个总线或互联网络来访问一个公共的内存，即所有资源均是共享的
- ② 处理机间通讯可通过共享内存来进行,比通过通讯机制进行通讯要快得多
- ③ 32或64节点以内并行算法**scaleup**很好
- ④ 超过32或64节点以后**scaleup**很坏，因为所有资源均是共享的，总线或互联网络就变成了一个瓶颈。超过这个点后增加处理机节点个数没有任何用处，因为处理机不得不花更多的时间来等待总线并访问内存和磁盘



硬件体系结构

(2) 共享磁盘(SD)

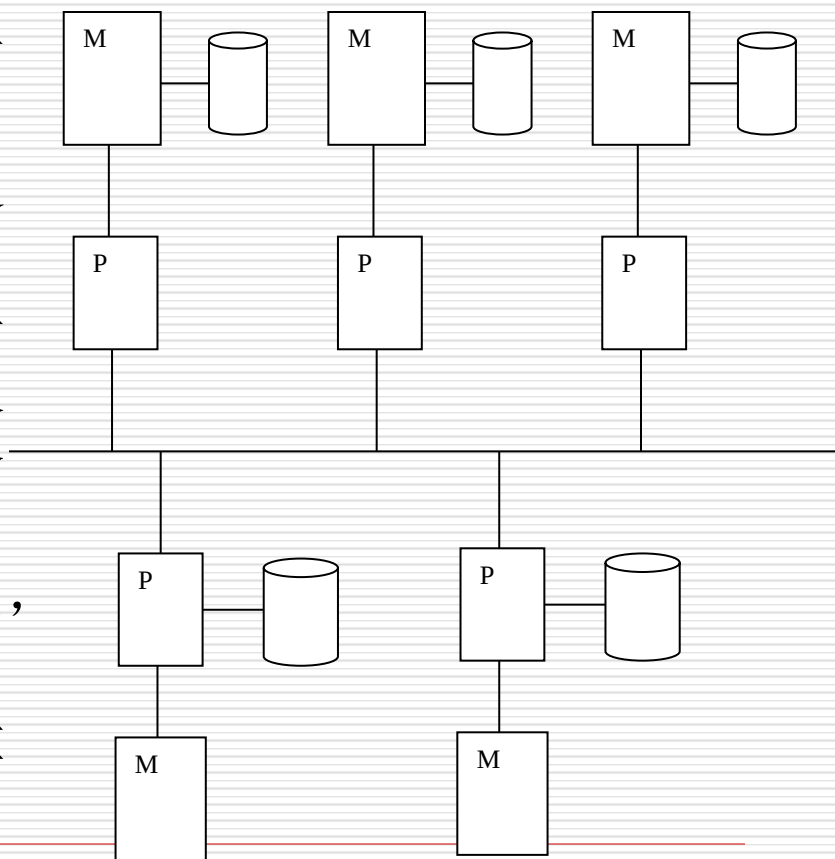
- ① 所有处理机可以直接通过总线或互联网络访问磁盘,但每个处理机有自己的私有内存
- ② 由于每个处理机有自己的内存, 存储器的总线不会成为瓶颈
- ③ 提供一定的容错能力, 若某处理机或它的内存出问题了, 其它处理机可以接管它的任务, 因为数据库驻留在所有处理机可以直接访问的磁盘上。磁盘子系统本身的容错问题可以通过使用**RAID**来解决
- ④ 尽管不存在内存共享, 共享磁盘仍然成为**SD**系统可扩展性问题的障碍, 共享的磁盘子系统的互联成为性能可扩展的瓶颈。**SD**不能解决可扩展性问题, 仅仅缓解了**SM**系统的可扩展性问题



硬件体系结构

(3) 无共享资源体系结构(SN)

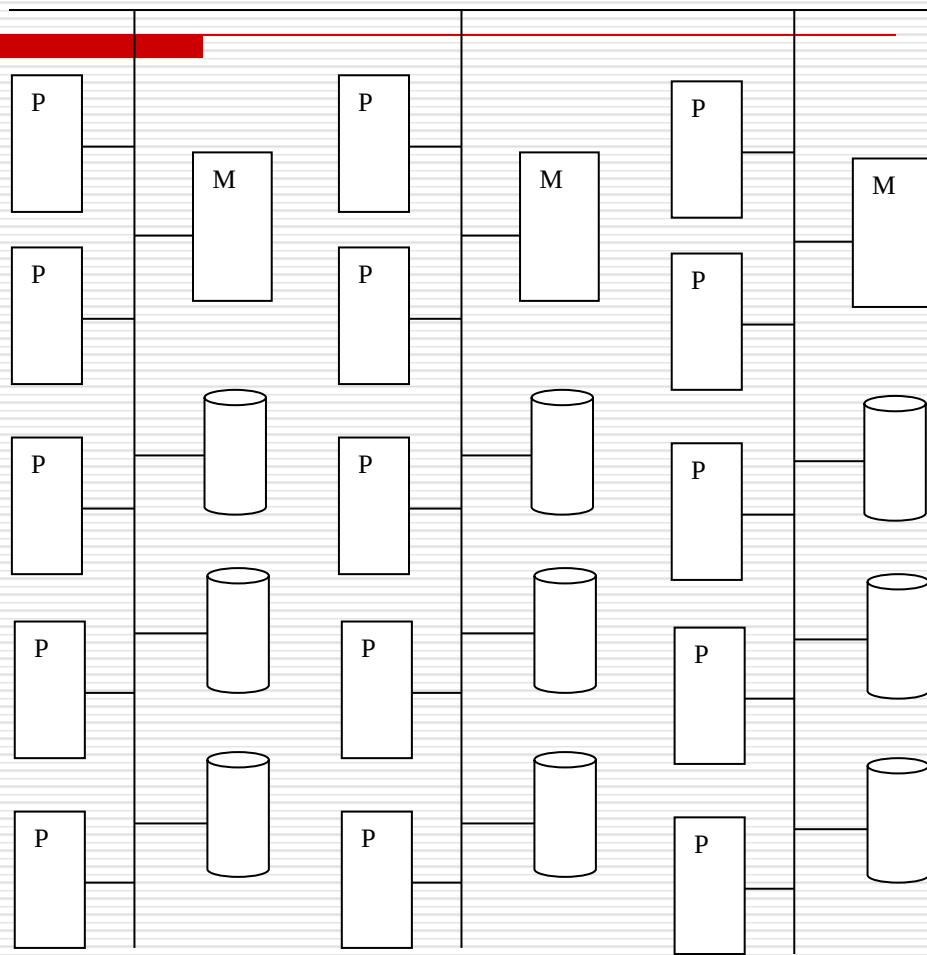
- ① 每个节点由一个处理机、内存和一个或多个磁盘构成
- ② 处理机之间通过高速互联网进行通讯
- ③ **SN**结构克服了**SD**结构必须通过一个总线进行**I/O**操作的缺点，仅仅是对非局部磁盘的存取才通过网络来进行。
- ④ **SN**体系结构具有很好的可扩展性，甚至可以扩展到成千上万个节点
- ⑤ 主要缺点是通讯代价和非局部磁盘的存取代价比较昂贵

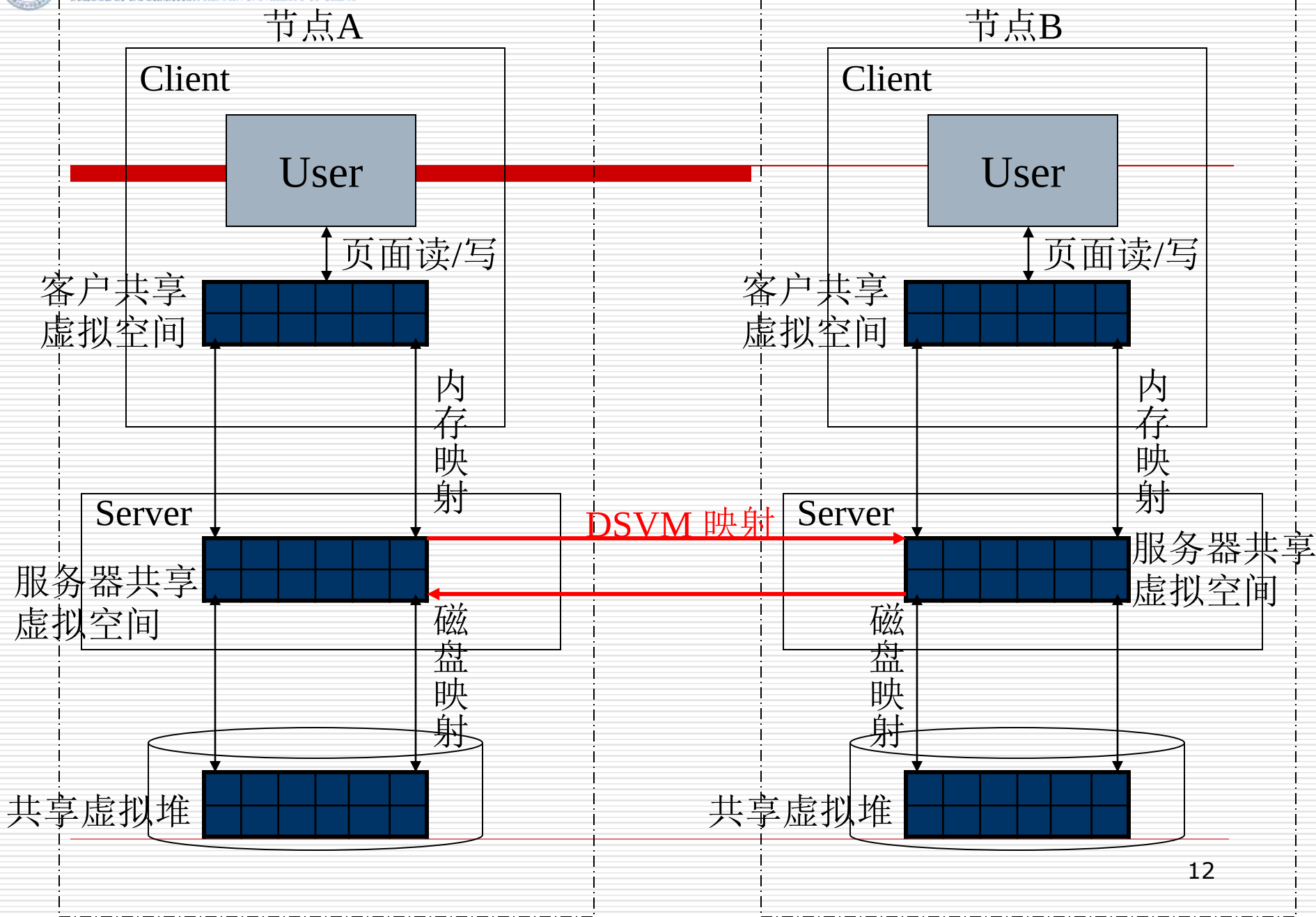


硬件体系结构

(4) 层次体系结构(COW, NOW)

- ① 结合了SN、SM、SD体系结构的特点，在高层看是一个SN体系结构，但每个节点是由一个SM体系结构所构成的。当然每个节点也可是一个SD体系结构
- ② 在这种体系结构中代码的编写是非常复杂的，降低编程复杂度的一种很好的办法是分布式虚拟存储器体系结构



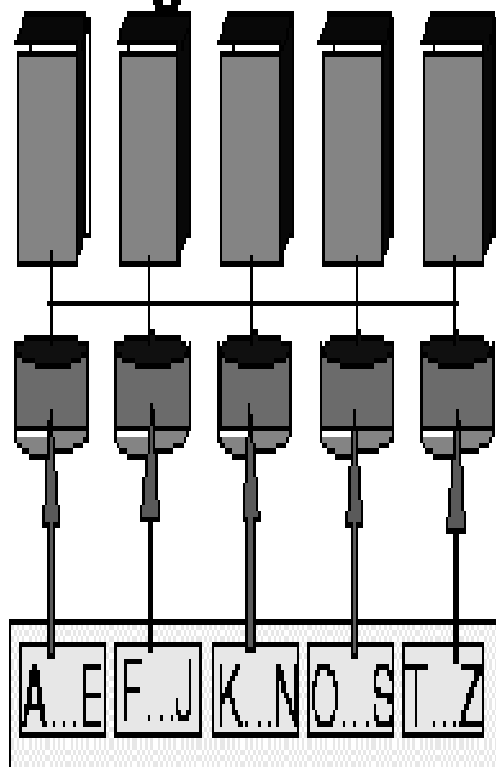


数据分片技术

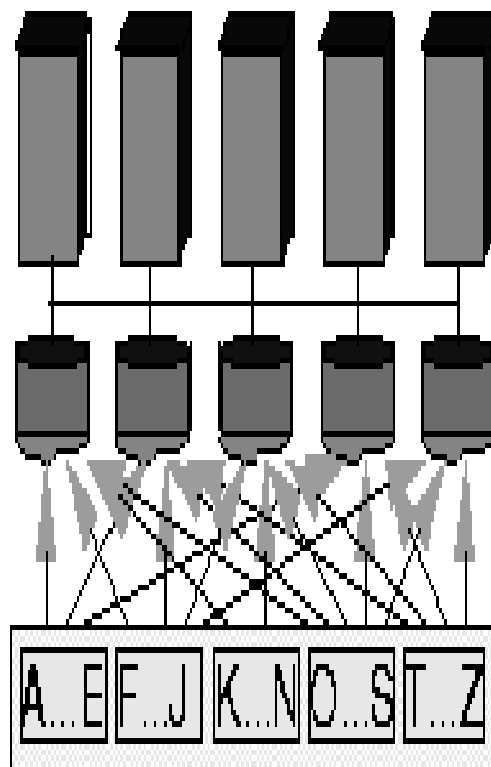
- (1) Round-robin: 对于关系 r 中的第 i 个元组分配到第 $(i \bmod n)$ 个磁盘上。该方法保证了每个磁盘上具有相同数目的元组数。
- (2) 哈希分片: 关系 r 中的一个或多个属性作为分片属性, 对于 r 中的元组 r_t , 该元组被分配到第 $h(r_t)$ ($0..n-1$) 个磁盘上。
- (3) 范围分片: 对于关系 r , 分片属性为 A , 则在 A 上可以定义一个分片向量: $[v_0, v_1, \dots, v_{n-2}]$ 。分片过程如下:
 - 若 $t[A] < v_0$, 则 t 被分配给第0个磁盘;
 - 若 $t[A] \geq v_{n-2}$, 则 t 分配给第 $n-1$ 个磁盘;
 - 若 $v_i \leq t[A] < v_{i+1}$, 则 t 被分配给第 $i+1$ 个磁盘。

Partitioning a table:

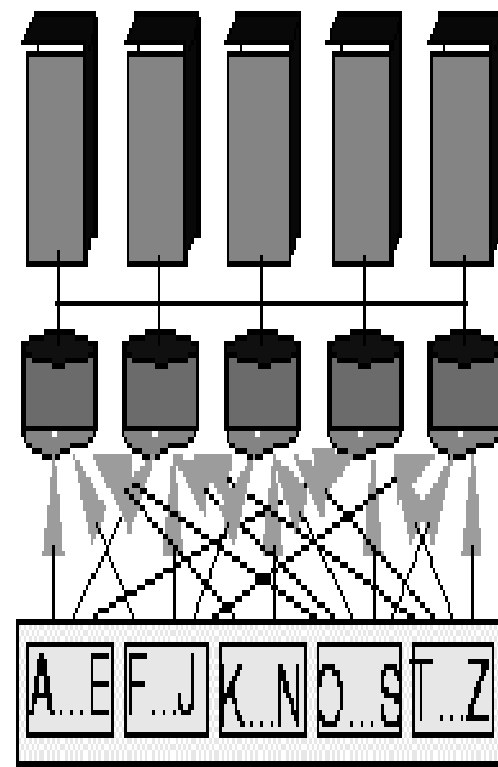
Range



Hash



Round Robin



数据分片技术

- 三种操作

- (1) 扫描整个关系

- (2) 点查询：如`employee-name="Campbell"`

- (3) 范围查询：`10000 < salary < 20000`

- (1) Round-robin：对于扫描操作非常好，但对于点操作和范围操作却不是很好

- (2) 哈希分片：对于基于分片属性的点操作是最好的。如果哈希函数能够保持随机性和均匀性，则哈希分片也能很好的处理扫描操作。但哈希分片方法不能很好地支持范围查询和基于非分片属性的点查询。

数据分片技术

- (3) 范围分片：能够很好地支持基于分片属性的点查询和范围查询。但这种支持既具有优点，也具有缺点。优点是：当一个范围查询只涉及到某几个磁盘时，该查询不必向其他磁盘发出查询请求，这样其他的磁盘可以响应其他的查询请求，提高了系统的吞吐量；缺点是：当在某几个磁盘上要存取大量的元组时，I/O成为瓶颈，造成执行倾斜，从而使得该查询的响应时间过长。
- (4) 除了round-robin分片处理以外，其他两种分片方法均可能造成倾斜问题。倾斜的分类：
- 属性值倾斜：属性值倾斜指的是很多元组在分片属性值上具有相同的元组，这必将导致倾斜。属性值分片将会导致分片倾斜无论采用范围分片还是哈希分片。
 - 分片倾斜：分片倾斜指的是在每个片段中的元组个数不同，即使不存在属性值倾斜问题也可能出现分片倾斜问题。



并行性种类

(1) 操作内并行性

- 多台机器同时执行某个操作(分片技术)

(2) 操作间并行性

- 多个操作并发地运行在多台机器上(管道技术)

(3) 查询间并行性

- 不同的查询运行在不同的机器上
- 主要讨论操作内并行性



并行连接

可分为以下**5**种连接：

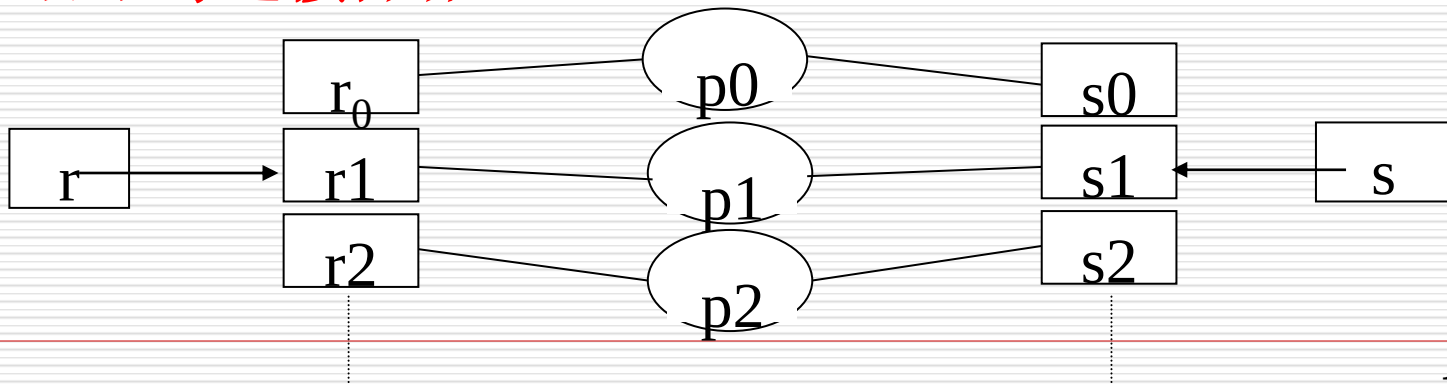
- ❑ 分片连接
- ❑ 非对称分片复制连接
- ❑ 对称式分片复制连接
- ❑ 并行哈希连接
- ❑ 并行嵌套循环连接

并行连接

□ 分片连接

- (1) 通过范围分片或哈希分片方法将 r 分片为 n 个片段,
 $r \rightarrow r_0, r_1, \dots, r_{n-1}$
- (2) 通过范围分片或哈希分片方法将 s 分片为 n 个片段,
 $s \rightarrow s_0, s_1, \dots, s_{n-1}$
- (3) 在处理器 p_i 上做子连接操作: $r_i \bowtie s_i$

仅适合于等连接操作

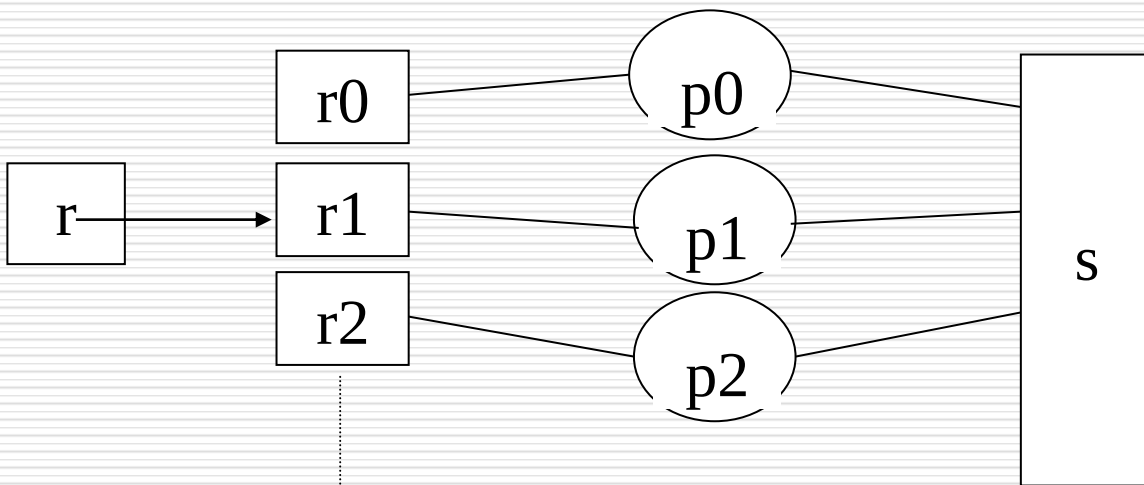


并行连接

□ 非对称分片复制连接

- (1) 可使用任何分片方法(包括round-robin)来将 r 分为 n 片
- (2) 将关系 s 复制到所有的处理机上
- (3) 处理机 p_i 执行子连接操作 $r_i \bowtie s$

适合任何形式的连接操作

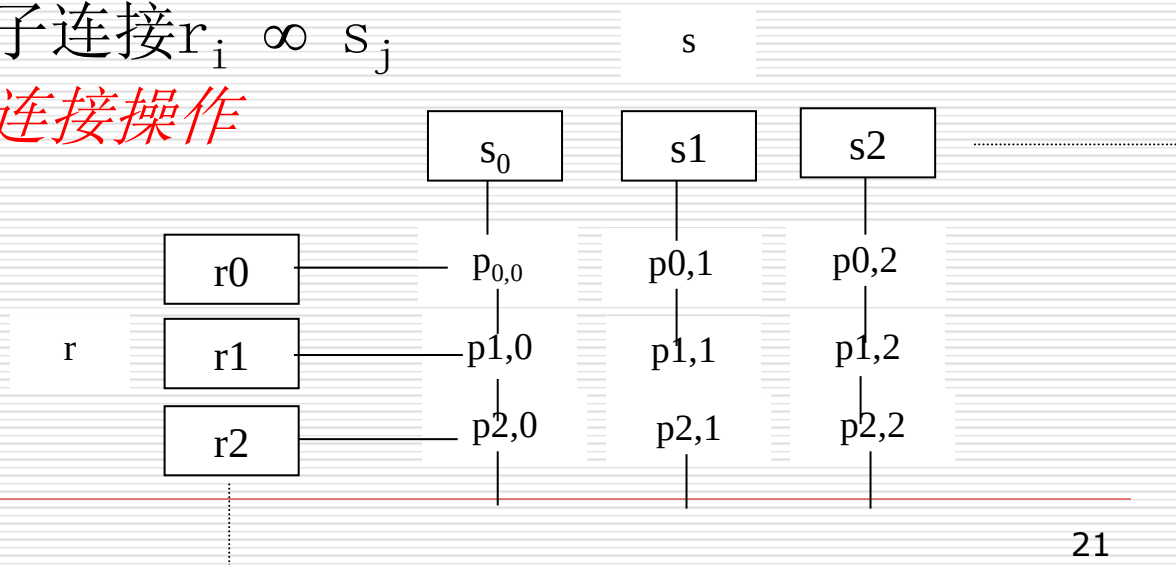


并行连接

□ 对称式分片复制连接

- (1) 将关系 r 分片为 m_1 片: $r \rightarrow r_0, r_1, \dots, r_{m_1-1}$
- (2) 将关系 s 分片为 m_2 片: $s \rightarrow s_0, s_1, \dots, s_{m_2-1}$
- (3) 将分片 r_i 发送到处理机 $P_{i,0}, P_{i,1}, \dots, P_{i,m_2-1}$
- (4) 将分片 s_i 发送到处理机 $P_{0,i}, P_{1,i}, \dots, P_{m_1-1,i}$
- (5) 处理机 $p_{i,j}$ 做子连接 $r_i \bowtie s_j$

适合任何形式的连接操作



并行连接

□ 并行哈希连接

- (1) 重分片 r : 在每个处理机 p_i 上, 对于磁盘 D_i 上 r 的每个元组 t_r , 计算 $h_1(\text{tr}[\text{JoinAttrs}])=j$, 则将该元组发送到处理机 P_j , 并形成关系 r 在处理机 P_j 上的再分片 r_j 。当所有的分片操作完成后在每个处理机上就能得到一个 r 的再分片。用相同的哈希函数再分片关系 s 。
- (2) 重分片 s : 与(1)相同
- (3) 在每个处理机 p_i 上哈希计算子连接 $r_i \bowtie s_i$

并行连接

- 并行嵌套循环连接
- 考虑关系 $s \lt r$, 对 r 进行分片存储, r 的每一个分区上存在连接属性上的一个索引。
 - (1) 采用非对称的分片-复制方法, 将关系 s 进行复制, 利用 r 现有的分区。
 - (2) 存放关系 s 的分区的每一处理器 P_i 对存放在 D_i 上的关系 s 中的元组进行读取, 并将其复制到其余的每一个处理器 P_j 中, 最后, 关系 s 被复制到存放关系 r 的元组的所有站点中。
 - (3) 在每个处理机 P_i 上对关系 r 的第 i 个分区做索引嵌套循环连接 $r_i \bowtie s_i$ 。通过将索引嵌套循环连接与对关系 s 的元组的划分并行操作, 减少将 s 元组写回磁盘再读取的代价。

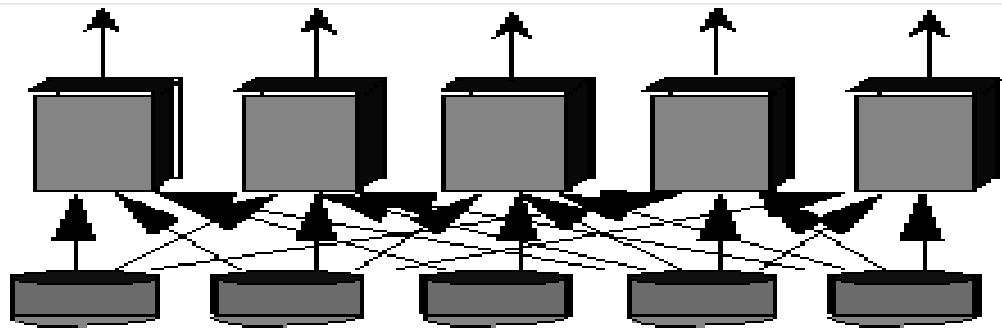


并行排序

- 假定被排序的关系被分放在 n 个磁盘上，可以采用两种方法来进行排序
 - (1) 并行范围分片排序
 - (2) 并行外部排序

并行排序

□ 并行范围分片排序



- 假定用 m 个处理机来排序具有 n 个分片的关系， $n > m$
- (1) 使用一个范围分片策略来重分片被排序的关系，使得在范围 i 上的元组被发送给处理机 P_i ，并将新的分片临时保存在磁盘 D_i 上。该步是并行执行的，有I/O开销和网络通讯开销
- (2) 处理机 P_i 排序存储在磁盘 D_i 上的分片 R_i ，
- (3) 合并操作：由于使用的是范围分片，合并操作相当简单，若 $i < j$ ，则处理机 P_i 上的元组关键字值小于处理机 P_j 上的元组关键字值

并行排序

□ 并行外部排序

- (1) 每个处理机 P_i 外部排序存储在磁盘 D_i 上的数据，该步是并行执行的
- (2) 合并每个处理机上的局部排序结果：
 - ① 每个处理机上排序后的分片进一步被范围分片到 m 个处理机上，这些元组以排序序来发送。
 - ② 每个处理机当收到来自其他处理机上的元组时进行合并操作
 - ③ 某个处理机最后合并所有处理机上的合并结果，这个合并非常简单



并行聚合

□ 有以下4种聚合算法：

- (1) 集中式二阶段并行聚合算法
- (2) 层次合并的并行聚合算法
- (3) 基于散列的两阶段并行聚合算法
- (4) 再分布并行聚合算法

并行聚合

□ 集中式二阶段并行聚合算法

- (1) 局部聚合阶段：它是在各个节点上进行一个集中式聚合操作
- (2) 集中合并阶段：各个局部聚合结果都发送到一个中央节点（称为协调者）进行最后的全局合并并生成最终的全局聚合结果
- 优点：当Group By子句的选择率比较小的时候，由于各个局部聚合结果的聚合组数较少，这样就减少了算法的通信开销，从而提高了算法的性能。
- 缺点：但随着Group By子句选择率的不断增加，通信开销也将随之增加，而且由于局部聚合的聚合组数的增大，中央协调者的工作负载也就越来越重，并最终可能成为并行算法的性能瓶颈

并行聚合

□ 层次合并的并行聚合算法

- (1) 局部聚合阶段：与集中式二阶段并行聚合算法相类似
- (2) 集中合并阶段：与集中式二阶段并行聚合算法不同，不是将各个节点的聚合结果发送到一个中央协调者，而是分层次并行地进行部分聚合结果的合并，并得到中间合并结果，这些中间结果可能被进一步并行地合并为新的中间结果或者合并为一个全局聚合结果。
- 该算法在性能上作了改进，减轻了合并节点的工作负担，但它并不能最终解决性能瓶颈问题，因为当 **Group By** 子句的选择率足够大时，层次合并阶段亦会成为该算法的性能瓶颈，只是该算法性能瓶颈的出现比集中式二阶段并行聚合算法来得晚些

并行聚合

□ 基于散列的两阶段并行聚合算法

- (1) **局部聚合阶段**：在这个阶段各个局部节点在各自的节点上进行集中式的聚合操作，并将局部聚合结果通过选择一个哈希函数作用到**Group By**属性上将局部聚合结果散列到各个节点上
- (2) **全局聚合阶段**：这个阶段主要是合并被散列到该节点上的局部聚合结果。由于对局部聚合结果进行了重新散列，所以全局聚合阶段的聚合结果不需要进行合并，这样全局聚合阶段是在各个节点上并行进行的，避免了前两种算法的性能瓶颈问题

并行聚合

□ 再分布并行聚合算法

- **基本思想：**先将 r 的每个子分片通过选择一个哈希函数并作用到**Group By**属性值上，将这些元组再分布到不同的节点上，在每个节点上有一个聚合操作，该操作接收再分布操作符发送过来的元组并进行聚合操作。由于对来自于聚合操作组的输入元组直接进行了散列，所以各个节点的聚合操作结果不需要进行合并



第十章 结束



考试重点

- **分片设计**: 分片类型、正确性判断、分片树
- **查询优化**: 全局**SQL**语句、关系代数表达式、查询树、优化后查询树、分片查询树（转换过程）
- **存取优化**: 计算全连接/半连接优化等策略的传输代价、执行场地情况
- **分布式事务**: 描述分布式事务执行过程
- **恢复管理**: 根据日志信息确定事务执行过程和**undo/redo**事务集
- **并发控制**: 判断历程可串行情况
- **数据复制**: **NWR**协议的一致性保证